Knowledge Distillation and Cloud-Native Deployment of a Lightweight Command-Parsing Language Model

Course: CS3081 Cloud Computing | **Student:** Sabavat Sachith Naik (Roll: 230101093)
GitHub Repository: https://github.com/Nani779-hub/230101093-cloud-miniproject

Executive Summary: This project delivers an end-to-end, resource-constrained sequence-level knowledge distillation pipeline and production-grade cloud-native microservice architecture. A high-parameter teacher model (Phi-3.5-mini-instruct, 3.8B parameters) was leveraged to generate semantic parse representations (intent and slots) for 3,000 commands drawn from the SLURP benchmark. Teacher outputs were parallel-ingested via sharded Kubernetes workers into SeaweedFS S3-compatible object storage, cleaned, and filtered. The resulting dataset was utilized to fine-tune an EleutherAI/Pythia-70M student model on pure CPU hardware. The student model was subsequently containerized into a minimal (<1.65 GB) FastAPI service, deployed on a multi-replica local k3d Kubernetes cluster with HPA autoscaling, zero-downtime rolling updates, health probes, and stateless Network Access Control Lists (NACLs), rigorously validated under CI/CD, and benchmarked under Locust load testing.

1. Task 1: Object Storage and Parallel Teacher Labelling

Architecture & Implementation: A local lightweight Kubernetes cluster was deployed via cluster/k3d.yaml with host-forwarded ports for HTTP/HTTPS ingress (8080) and object storage. Inside the data namespace, SeaweedFS was provisioned providing S3-compliant object storage with buckets datasets and models. The teacher model (Phi-3.5-mini, 3.8B parameters) was hosted locally via Ollama with fixed hyperparameters (temperature=0, seed=42) to guarantee deterministic token generation. A system prompt (prompts/teacher_prompt.txt) strictly constrained output to a four-key JSON frame: intent, date, time, and place.

Parallel Sharding Strategy: To distribute workload without inter-worker coordination or locking, deterministic modulus sharding was implemented: worker *k* out of *N* processes record *i* if i mod N == k. Two independent Kubernetes Jobs (teacher-worker-0 and teacher-worker-1) ran concurrently in the pipeline namespace, each processing 1,500 commands and checkpointing outputs to s3://datasets/teacher/shard-0.jsonl and shard-1.jsonl every 10 records.

Data Cleaning & Quality Gate (clean.py): Shards were merged and sanitized against criteria (a) through (e). The strict mathematical identity input = (a) + (b) + (c) + (d) + duplicates + kept (3000 = 1032 + 0 + 0 + 46 + 34 + 1888) was verified:

Category / Reason	Count	Percentage	Description / Action
Total Input Records	3,000	100.0%	Combined lines across shard-0 and shard-1
(a) Invalid JSON	1,032	34.4%	Model returned malformed syntax, truncated braces, or non-JSON
(b) Wrong Keys	0	0.0%	Missing required keys or containing unexpected extraneous keys
(c) Invalid Intent	0	0.0%	Intent outside the eight sanctioned SLURP intents
(d) Hallucinated Slots	46	1.5%	Slot text not verbatim present in natural language input command
(e) Duplicate Commands	34	1.1%	Redundant utterances; retained entry with lowest command ID
Valid Clean Records Kept	**1,888**	**62.9%**	**Preserved in ID order; saved to datasets/v1/train.jsonl**

SHA-256 Verification: Cleaned dataset checksum was computed and stored at datasets/v1/train.sha256 (a6ba785ca8cc3ab5dfe853cd01371178ce265f383279cf176ba99463656ea6ad). Re-running clean.py on raw shard outputs deterministically reproduces this checksum.

2. Task 2: Training the Student (Sequence-Level Distillation)

Motivation & Distillation Theory: Deploying modern LLMs (>3B parameters) on low-power edge nodes or high-concurrency microservices induces prohibitive memory footprints and latency bottlenecks. Sequence-Level Knowledge Distillation (Kim & Rush, EMNLP 2016) mitigates this by training a compact student model (Pythia-70M, 70M parameters) directly on teacher-generated natural language token sequences: Command: <text>\nJSON: <teacher_reply>. In contrast to token-level or logit-based distillation (which minimizes Kullback-Leibler divergence over full teacher softmax distributions P_T(y_t | y_{<t}, x)), sequence-level distillation requires only the teacher's greedily decoded or sampled surface outputs y* = argmax_y P_T(y|x).

Why Logit-Based Distillation is Infeasible Here: (1) The teacher model (Phi-3.5, vocabulary size 32,064) and student model (Pythia-70M, vocabulary size 50,304) possess non-aligned token vocabularies and differing subword tokenizers (Llama/TikToken BPE vs. GPT-NeoX BPE), making direct per-token cross-entropy or logit alignment mathematically non-trivial. (2) Standard inference runtimes (such as Ollama REST API) expose generated text tokens rather than raw logit distribution tensors across all 32,000+ dimensions. (3) Sequence-level cross-entropy loss on teacher outputs avoids the massive memory transfer overhead of saving full probability distributions.

Implementation & Hyperparameters: As specified in training/config.yaml, training was executed on CPU using PyTorch and Hugging Face Transformers with seed 42, AdamW optimizer (learning rate 5.0e-5, weight decay 0.01), batch size 8, sequence length 80, linear warmup, and gradient clipping at 1.0. Version 1 was trained for 5 epochs (loss decreased from 2.2158 to 0.2541), and Version 2 was trained for 7 epochs (loss reached 0.1716). Both versions were uploaded to SeaweedFS under models/v1/ and models/v2/.

Empirical Evaluation on validation.jsonl (100 Commands):

Model	Parameters	JSON Validity	Intent Acc.	Date Acc.	Time Acc.	Place Acc.	Full Frame
Teacher (Phi-3.5-mini)	3.8 B	70.0%	62.0%	56.0%	59.0%	59.0%	**33.0%**
Untrained Student (Pythia-70M)	70 M	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%
Trained Student v1 (5 epochs)	70 M	**97.0%**	**74.0%**	39.0%	50.0%	61.0%	**29.0%**
Trained Student v2 (7 epochs)	70 M	97.0%	75.0%	40.0%	51.0%	62.0%	30.0%

Analysis & Limitations: Untrained Pythia-70M produces 0% valid JSON, as it lacks domain knowledge of structured syntax. Post-distillation, JSON validity reaches **97.0%** (exceeding the 95% quality gate), and full-frame accuracy reaches **29.0%**, comfortably exceeding half of the teacher's full-frame score (33.0% / 2 = 16.5%), satisfying all grading milestones. The core limitation of sequence distillation is error propagation: when the teacher mislabels an input, the student faithfully memorizes the incorrect target.

3. Task 3: The Inference Service in a Container

Service Architecture: Implemented in service/app.py with FastAPI and Uvicorn. Exposes three core endpoints: (1) POST /parse parsing natural language commands into 4-key JSON with response headers X-Model-Version and X-Served-By; (2) GET /healthz returning HTTP 200 for liveness; (3) GET /readyz returning HTTP 503 during model download and HTTP 200 once weights are fully loaded into memory.

Externalized Weight Retrieval & Secret Ingestion: Image layers strictly exclude model weights. On startup, the service queries S3 using Boto3 configured with environment variables (MODEL_URI, MODEL_VERSION, S3_ENDPOINT, MAX_NEW_TOKENS) and credentials populated exclusively from a Kubernetes Secret (s3-credentials).

Schema Validation & Fault Injection: Generated outputs are validated against cloud_assignment/data/schema.json. Malformed JSON or schema deviations trigger HTTP 500 with {"error": "invalid_model_output"}. With FAULT_INJECT=1, model output is overridden with "not json", validating fail-safe error handling.

Container Optimization & Resource Limit Verification: Built via multi-stage Dockerfile utilizing CPU-only PyTorch (torch==2.6.0+cpu), stripping CUDA/cuDNN binaries. Final image disk footprint is **1.64 GB** (under the 2.0 GB ceiling). Running 50 sequential requests under cgroup limits of 1 vCPU and 1 GiB memory yielded a measured peak memory of **333.9 MiB (0.326 GiB)** with zero OOM events.

4. Task 4: Kubernetes Deployment, Scaling, and Reliability

Cluster Topology & Ingress Routing: Deployed to namespace app on k3d Kubernetes. Ingress routing maps port 8080 traffic to the ClusterIP Service on port 80, forwarding to pod port 8000. Traefik handles Layer 7 routing and round-robin load distribution.

Horizontal Pod Autoscaling (HPA): Configured with min 1, max 2 replicas, target CPU utilization 70%. Under Locust load test, metrics-server registered >70% CPU, automatically triggering scaling from 1 to 2 replicas within 45 seconds, and scaling back down post-cooldown.

Rolling Updates & Rollback: During active traffic, deployment images were updated from v1 to v2 using maxUnavailable: 0 and maxSurge: 1. A total of 232 requests were served across both versions (112 on v1, 120 on v2) with zero network downtime. A subsequent kubectl rollout undo deployment/command-parser -n app successfully restored X-Model-Version: v1.

Pod Recovery & Cold Start: Deleting an active pod during load testing resulted in immediate replica replacement with only 1 dropped request out of 160 (**0.625%** error rate, well below the 2.0% threshold). Cold-start benchmarks (measuring duration from pod creation to readyz HTTP 200 across 3 clean runs) recorded **72.58s, 9.82s, and 19.02s** (median: **19.02s**).

Load Balancing Distribution: Locust header analysis across 2 healthy replicas over 2 minutes showed Replica A served **51.8% (57 requests)** and Replica B served **48.2% (53 requests)**, comfortably satisfying the 30% balance rule.

5. Task 5: Subnetting and Network Access Control (NACL)

Subnet Partitioning (10.20.0.0/16): Allocated consecutively from largest host requirement to smallest using minimal power-of-two prefixes 2^(32-prefix) - 2 >= Hosts:

Subnet	Hosts Req.	Network CIDR	Prefix	Usable Host Range	Broadcast	Usable Hosts
app	1,000	10.20.0.0	/22	10.20.0.1 – 10.20.3.254	10.20.3.255	1,022
public	250	10.20.4.0	/24	10.20.4.1 – 10.20.4.254	10.20.4.255	254
data	100	10.20.5.0	/25	10.20.5.1 – 10.20.5.126	10.20.5.127	126
management	20	10.20.5.128	/27	10.20.5.129 – 10.20.5.158	10.20.5.159	30

Stateless NACL Theory & Rule Ordering: Stateless NACLs do not preserve session state; each inbound and outbound packet is evaluated independently. Return web traffic requires explicit outbound ephemeral port rules (1024–65535). Rules are evaluated in strict ascending order: rule 100 explicitly blocks abusive range 198.51.100.0/24 across all ports before rule 110/120 allows broad 0.0.0.0/0 traffic. Verified 6/6 (100%) via tools/nacl_sim.py.

6. Task 6: Reproducibility, Testing, and CI/CD

One-Command Lifecycle: make up creates the k3d cluster, provisions SeaweedFS, imports Docker images, and deploys services in under 3 commands. make down safely purges all resources.

GitHub Actions Pipeline (.github/workflows/ci.yml): Implemented with 4 sequential stages: (1) lint (flake8 on all directories, 0 errors); (2) test (23 pytest unit tests covering cleaning, schema, validation, and NACL rules); (3) evaluate (downloads model from GitHub Release, executes score.py, enforces quality gate --min-validity 95); (4) secret-scan (Gitleaks automated scan reporting 0 leaks).

Quality Gate Failure Verification: When triggered with the untrained Pythia-70M model, the quality gate accurately failed with json_validity: 0.0% < 95.0%, halting the pipeline and demonstrating gate integrity.

7. Task 7: Performance, Cost, and Comprehensive Review

Locust Load Test Measurements (2 minutes, 4 users): p50 latency: **2,800 ms**, p95 latency: **9,000 ms**, throughput: **0.95 req/s**, total requests: **113**, error rate: **2.65% (3 schema rejections, 0 timeouts, 0 crashes)**, pod CPU under load: **554m and 549m**, pod memory: **349 MiB and 362 MiB**.

Cost Estimation (rate_card.json): Hourly cost per replica = (1 vCPU * 3.50 INR) + (1 GiB * 0.45 INR) = **3.95 INR/hr**. For 2 running replicas: **7.90 INR/hr**. Requests served per hour = 110 successful req / 120s * 3600s = **3,300 req/hr**. Cost per 1,000 requests = (7.90 / 3,300) * 1,000 = **2.39 INR**.

SLO Definition & Error Budget: Target: **95.0%** HTTP 200 responses over 2-minute load window (113 total requests). Allowed error budget: **5.65 requests**. Actual errors observed: **3 requests** (53.1% of budget consumed, 46.9% remaining). Measured success rate: **97.35%**.

Critical Review Table (6 Dimensions):

Component / Concern	Evidence (Measured Value)	Limitation Identified	Proposed Improvement
Data & Storage	1,888 kept records; 34 duplicates removed; 19.02s median cold start	S3 object retrieval on pod startup introduces ~19s init delay	Pre-warm models via local node SSD DaemonSet caching
Model Quality	97.0% JSON validity; 29.0% full frame vs 33.0% teacher	Teacher labeling errors are propagated and memorized	Multi-teacher ensemble distillation with majority voting
Latency & Throughput	p50: 2,800 ms; min: 595 ms; throughput: 0.95 req/s	CPU token autoregression limits concurrency scaling	Quantize to INT8/INT4 (ONNX/vLLM) to double throughput
Resource Utilisation	333.9 MiB peak RAM; 554m CPU under 1 core / 1 GiB limit	Static 1 GiB limit causes headroom underutilization	Tune limits to 512 MiB with VPA memory recommendation
Reliability & Recovery	0.0% rolling update downtime; 0.625% pod recovery error	Single-node laptop cluster lacks multi-host fault tolerance	Deploy multi-zone topology with pod anti-affinity rules
Security & Cost	2.39 INR / 1k req; 0 gitleaks detected; 6/6 NACL passed	Stateless NACLs require broad ephemeral return port ranges	Combine NACLs with stateful Cilium / Calico NetworkPolicies

Key References: (1) Kim & Rush, 'Sequence-Level Knowledge Distillation', EMNLP 2016; (2) Bastianelli et al., 'SLURP: A Spoken Language Understanding Resource Package', EMNLP 2020; (3) Biderman et al., 'Pythia: A Suite for Analyzing Large Language Models', ICML 2023; (4) Google SRE Book, 'Service Level Objectives', O'Reilly Media; (5) Kubernetes Networking & Ingress Architecture Guide.